Can LLMs Design Video Coding Tools? A Case Study on Planar Mode

Yingwen Zhang[1], Meng Wang[2], Liqiang He[1], Shiqi Wang[1‡]
[1]Department of Computer Science, City University of Hong Kong
[2]School of Data Science, Lingnan University

arXiv:2609.01535v2 [cs.MM] 27 Oct 2026

***Abstract*—This paper explores whether large language models (LLMs) can design video coding tools, a highly challenging task due to the intricate algorithmic coupling of tool modifications. In particular, we present an empirical case study on the Planar mode, a long-standing intra prediction tool in video coding standards. Our experiments operate within a generation-and-evaluation loop, with the LLM generating new Planar predictors, encoder trials evaluating their coding performance, and the LLM re-generating refined implementations based on the evaluation feedback. We first examine directly replacing the default Planar mode in the Fraunhofer Versatile Video Encoder (VVenC) under its *faster* preset. Experimental results demonstrate that the LLM-generated mode can outperform the conventional Planar mode on this lightweight toolset, achieving 0.18% bitrate savings with 0.4% complexity overhead on the standard benchmark. We further extend our evaluation to the Enhanced Compression Model (ECM). Leveraging newly introduced directional Planar modes, we investigate two integration strategies: directly replacing them, and introducing the LLM-generated predictor as an additional prediction mode with new syntax elements. The empirical results suggest that both strategies can yield coding gains under a constrained low-resolution setting. Overall, this study offers preliminary evidence and practical insights, highlighting both the potential and open challenges of LLM-based coding tool design.**

***Index Terms*—video coding, intra prediction, large language models, automatic algorithm design**

I. Introduction

Large language models (LLMs) have recently expanded from general question-answering tasks to more structured applications in automatic algorithm design [1]–[3]. In video coding, this paradigm has shown promising results in quantization parameter (QP) allocation [4]. Typically, an evolutionary generation-and-evaluation loop is constructed, where the LLM acts as a designer to iteratively propose QP decision rules, actual encoder executions evaluate these rules to compute BD-rate [5] performance, and the resulting feedback guides the subsequent refinement of the algorithm. This line of research demonstrates that LLMs, when guided by proper encoding feedback, can produce effective code-level solutions for well-defined encoder optimization tasks. A critical next question is whether this approach can be extended to designing video coding tools themselves.

From Advanced Video Coding (AVC) [6] to Versatile Video Coding (VVC) [7] and toward the emerging next-generation standard [8], coding gains have largely resulted from the iterative proposal, validation, and refinement of coding tools. Meanwhile, the coding tool development remains heavily expert-driven: domain experts conceive an idea, integrate it into reference software, evaluate its coding efficiency and complexity, and iteratively improve it before standardization discussion. While effective, the explored design space is inevitably limited by human domain knowledge. LLMs provide a new opportunity to expand this search space. However, unlike a QP allocation rule [4] applied at the pre-encoding stage, a coding tool is deeply embedded in the core codec pipeline. Consequently, its modifications introduce an intricate algorithmic coupling with surrounding modules. For example, modifications to a prediction tool may propagate a cascading impact to downstream stages such as transform, quantization, and rate-distortion optimization (RDO). Furthermore, any logic adjustment must carefully account for syntax signaling overhead and strict encoder-decoder matching. These stringent requirements make coding tool design significantly more challenging for LLMs.

In this paper, we conduct a case study on the Planar mode [9], a foundational intra prediction tool across multiple video coding standards. Its basic form is a linear interpolation from reconstructed boundary samples, making it a simple and interpretable starting point for investigating LLM-based tool design. We ask a narrow question: under existing codec syntax and interface constraints, can an LLM generate a Planar function that outperforms the hand-crafted baseline in certain configurations? To answer this, we explore the direct mode replacement in the Fraunhofer Versatile Video Encoder (VVenC) [10] *faster* preset, and then further evaluate both direct replacement and new-mode insertion in the Enhanced Compression Model (ECM) [11] based on the directional Planar mode [12] implementation. This work represents a first step toward examining where LLM-based coding tool design shows promise and what remains open.

II. Methodology

A. Preliminaries: Planar Prediction

Planar prediction [9] was introduced to approximate smoothly varying two-dimensional surfaces. Rather than explicitly signaling spatial control points, it interpolates the lower-right texture region by fully leveraging the reconstructed boundary samples. Formally, given a coding block of width W and height H, the prediction value $\hat{P}(x, y)$ within $0 \leq x < W$ and $0 \leq y < H$ is defined as a linear combination of horizontal and vertical interpolations. While the left boundary $L(-1, y)$

‡ Corresponding author

and top boundary $T(x,-1)$ represent readily available reconstructed samples, the virtual bottom-right references are extrapolated from the bottom-left corner $L(-1,H)$ and top-right corner $T(W,-1)$, respectively. The resulting mathematical formulation is expressed as

$$\hat{P}(x,y) = \frac{HF_{\mathrm{h}}(x,y) + WF_{\mathrm{v}}(x,y)}{2WH}, \tag{1}$$

where the horizontal and vertical interpolation components, $F_{\mathrm{h}}(x,y)$ and $F_{\mathrm{v}}(x,y)$, are defined as:

$$\begin{aligned} F_{\mathrm{h}}(x,y) &= (W-1-x)L(-1,y) + (x+1)T(W,-1), \\ F_{\mathrm{v}}(x,y) &= (H-1-y)T(x,-1) + (y+1)L(-1,H). \end{aligned} \tag{2}$$

Owing to its high coding efficiency for homogeneous regions, Planar prediction serves as a foundational intra tool and is commonly prioritized in the construction of the most-probable-mode (MPM) list [13].

Recent ECM development further extends this family with directional Planar variants that retain only one interpolation direction [12]. Let $\hat{P}_{\mathrm{h}}(x,y)$ and $\hat{P}_{\mathrm{v}}(x,y)$ denote the predicted sample values of the horizontal and vertical directional Planar modes, respectively. They can be computed as

$$\begin{aligned} \hat{P}_{\mathrm{h}}(x,y) &= \frac{1}{W} \cdot F_{\mathrm{h}}(x,y), \\ \hat{P}_{\mathrm{v}}(x,y) &= \frac{1}{H} \cdot F_{\mathrm{v}}(x,y). \end{aligned} \tag{3}$$

These two variants are pre-selected by the Hadamard cost [14] and then inserted into the full RDO list when competitive. If a directional Planar mode is used, it is signaled through the Planar branch of the MPM syntax, with an index distinguishing the original, horizontal, and vertical variants. The transform side is also adapted to the directional bias: the horizontal and vertical Planar variants use the corresponding horizontal and vertical intra modes to derive the transform kernels [12].

B. LLM-Based Planar Mode Design

The core of the Planar-family mode is a prediction function $f(\cdot)$ that maps reconstructed boundaries to a two-dimensional prediction block. In this study, we examine whether such prediction functions can be automatically designed by LLMs. Our workflow follows an iterative generation-and-evaluation loop: given a collection of historical candidates and their coding feedback, the LLM generates novel, codec-compliant C++ code of $f(\cdot)$ alongside the underlying design ideas. Importantly, the search target of LLMs is deliberately limited to the core predictor body, while the surrounding codebase such as reference preparation remains unchanged. Each generated candidate is then syntactically validated, compiled into the codec, and evaluated through actual encoding runs. The measured coding performance is subsequently fed back into the next prompting iteration, allowing the workflow to accumulate this empirical history and iteratively search for better predictor formulations.

Herein, to ensure that the generated Planar predictors remain structurally lightweight and compatible with codec implementation norms, we simply enforce these operational boundaries

Task description. Optimize the prediction rule used by the original Planar mode in VVenC. Given the prepared top and left reference lines for the current block, generate a deterministic intra prediction rule that fills the whole block. Find practical and stable coding-gain improvements under strict encoder-decoder matching.

Historical feedback. (Inserted multiple parent predictors, including a short design idea, their raw C++ body, and per-sequence BD-rate feedback).

Strategy-specific instruction. Create a new predictor that is structurally different from the given ones. Keep the new design in the same or lower complexity class as the parents. First, write exactly one sentence inside `<idea>...</idea>` that describes the new algorithm at a high level. Next, write exactly the code block inside ```` ```cpp``` ````.

Interface specification. The generated C++ body is inserted inside `xPredIntraPlanar_Core`. The surrounding function has already prepared `width`, `height`, `stride`, `pred`, `top[]`, and `left[]`. The generated code must write every sample through `pred[y * stride + x]`.

Validity constraints. Use deterministic integer arithmetic and valid boundary indexing. Keep the main prediction loop lightweight; statistics precomputation is allowed when useful. Do not use encoder-only state, randomness, dynamic allocation, I/O, preprocessor directives, or external helper logic.

Fig. 1. A compact (exploration) prompt example for VVenC Planar design.

through prompt engineering. Each prompt is structured around five core components. First, the task description explicitly defines the target objective as designing novel Planar prediction mechanisms. Second, the historical feedback component introduces parent candidates from prior iterations; crucially, each parent candidate encapsulates both its conceptual design idea and its corresponding C++ code, paired with its sequence-level BD-rate performance. Third, the strategy-specific instruction implements two evolutionary search strategies [2]: exploration instructions direct the LLM to synthesize predictors that are structurally distinct from existing parent candidates, whereas modification instructions guide the LLM to perform localized structural or numeric variations on a single parent candidate. Fourth, the interface specification strictly defines the accessible input variables and mandates that the generated code outputs the prediction block. Fifth, the validity constraints enforce that the proposed predictors are decoder-reproducible and computationally practical. Candidates that fail syntactic parsing, interface compliance, compilation, or decoding are rejected and not retained for the next iteration. Fig. 1 shows a compact prompt example for the VVenC Planar design used in our direct-replacement experiments.

After encoding, the generated predictors are merged into a historical candidate pool. We sort this pool according to an objective $\mathcal{J}$ and retain the top-K candidates as parents for subsequent prompts. Since the average BD-rate $\bar{\mathcal{B}}(f)$ alone can favor predictors that improve a few sequences while degrading others, we add a worst-case-aware regularization term $\mathcal{R}(f)$:

$$\mathcal{J}(f) = \bar{\mathcal{B}}(f) + \lambda \mathcal{R}(f), \tag{4}$$

where $\mathcal{R}(f)$ is the average BD-rate of the worst-M sequences, and λ controls the regularization strength. This objective is used for sorting and parent sampling. For each exploration prompt, we sample N parent candidates based on ranking, where candidates with lower rank indices are assigned higher probability. In our experiment, we fix $K = 16$, $M = 2$, $N = 8$, and generate 64 candidates for each iteration by repeating the exploration and modification prompts. The core LLM is DeepSeek-V4-Flash [15] with high thinking mode.

TABLE I
EVOLVED PLANAR RESULTS (ANCHOR: DEFAULT VVENC-1.14.0 *faster*).

	$\lambda = 2$		$\lambda = 5$	
Sequence	BD-rate	ΔEncT	BD-rate	ΔEncT
Tango2	−0.14%	+0.9%	−0.05%	+5.9%
FoodMarket4	−0.15%	+0.6%	−0.03%	+5.9%
Campfire	−0.23%	+0.8%	−0.16%	+4.4%
CatRobot	−0.14%	+1.1%	−0.12%	+4.5%
DaylightRoad2	−0.07%	+0.7%	−0.10%	+5.0%
ParkRunning3	−0.12%	0.0%	−0.07%	+3.6%
Class A Average	**−0.14%**	**+0.7%**	**−0.09%**	**+4.9%**
MarketPlace	−0.16%	+0.9%	−0.13%	+5.2%
RitualDance	−0.17%	+1.5%	−0.12%	+4.5%
Cactus	−0.22%	+0.1%	−0.16%	+3.5%
BasketballDrive	−0.16%	+1.2%	−0.13%	+3.8%
BQTerrace	−0.17%	+0.3%	−0.20%	+3.8%
Class B Average	**−0.18%**	**+0.8%**	**−0.15%**	**+4.2%**
BasketballDrill	−0.22%	−0.4%	−0.21%	+4.0%
BQMall	−0.11%	+0.4%	−0.16%	+4.6%
PartyScene	−0.34%	−0.8%	−0.27%	+3.0%
RaceHorsesC	−0.27%	0.0%	−0.22%	+4.0%
Class C Average	**−0.23%**	**−0.2%**	**−0.22%**	**+3.9%**
BasketballPass	−0.14%	+0.4%	−0.09%	+2.3%
BQSquare	−0.27%	−0.2%	−0.27%	+2.3%
BlowingBubbles	−0.32%	−0.1%	−0.27%	+2.5%
RaceHorses	−0.23%	−0.2%	−0.20%	+2.4%
Class D Average	**−0.24%**	**0.0%**	**−0.21%**	**+2.4%**
FourPeople	−0.06%	+1.4%	−0.18%	+4.7%
Johnny	−0.22%	+0.9%	−0.19%	+4.4%
KristenAndSara	−0.04%	+0.4%	−0.18%	+4.9%
Class E Average	**−0.11%**	**+0.9%**	**−0.18%**	**+4.7%**
ArenaOfValor	−0.20%	+0.4%	−0.24%	+4.3%
BasketballDrillText	−0.22%	+0.7%	−0.23%	+3.7%
SlideEditing	−0.08%	+0.7%	−0.04%	+1.5%
SlideShow	−0.16%	−0.3%	−0.13%	+2.0%
Class F Average	**−0.17%**	**+0.4%**	**−0.16%**	**+2.9%**
All Average	**−0.18%**	**+0.4%**	**−0.16%**	**+3.9%**

III. EXPERIMENTS

A. VVenC Mode Replacement

We first evaluate the direct Planar replacement in VVenC-1.14.0 [10] under the lightweight *faster* preset, where most of the VVC intra tools are disabled. Our experiment directly reuses the existing Planar syntax, RDO process, and prepared reference samples in VVenC. Only the core Planar predictor operating on these references is replaced by an LLM-generated one. As the all-intra (AI) configuration is not provided in VVenC, we modify the random access configuration to match the common test conditions (CTC). We turn on the Wavefront Parallel Processing (WPP) with four threads. The YUV-PSNR ratio is 6:1:1. In training, we use the default encoder as the baseline and evaluate the performance of the candidates on the first frame of the 22 natural sequences from CTC (Classes A to E). After 20 iterations of training, we test the candidates of the last iteration on full-length CTC sequences.

In Table I, coding performance of two evolved predictors at $\lambda = 2$ and $\lambda = 5$ is demonstrated. Both predictors achieve consistent bitrate savings over the VVenC anchor, with average BD-rate reductions of 0.18% and 0.16%, respectively. The $\lambda = 2$ candidate yields slightly larger coding gains with only a 0.4% time increase, while the $\lambda = 5$ candidate achieves a smaller BD-rate reduction. Interestingly, the two selected predictors correspond to two clearly different evolutionary directions. The $\lambda = 2$ predictor improves Planar mainly through adaptive boundary filtering: It first computes the gradient of the top and left reference lines as

$$\begin{aligned} V_T &= \sum_{i=1}^{W} |T(i,-1) - T(i-1,-1)|, \\ V_L &= \sum_{j=1}^{H} |L(-1,j) - L(-1,j-1)|, \end{aligned} \tag{5}$$

and then applies a three-tap smoothing filter with coefficients $[1, 2, 1]$ to both reference lines. If one side is much less smooth than the other, namely $V_T > 5V_L/4$ or $V_L > 5V_T/4$, the same filter is applied twice to that side. This gives

$$\tilde{T} = \begin{cases} \mathcal{S}(\mathcal{S}(T)), & V_T > 5V_L/4, \\ \mathcal{S}(T), & \text{otherwise}, \end{cases} \tag{6}$$

and $\tilde{L}$ is obtained symmetrically. Subsequently, the predictor applies Eqn. (2) to $\tilde{T}$ and $\tilde{L}$ to obtain the two interpolation components $\tilde{F}_{\mathrm{h}}$ and $\tilde{F}_{\mathrm{v}}$. Finally, this predictor tunes the last averaging step by a gradient-dependent offset ρ:

$$\hat{P}_{\lambda=2}(x,y) = \frac{H\tilde{F}_{\mathrm{h}}(x,y) + W\tilde{F}_{\mathrm{v}}(x,y) + \rho}{2WH}, \tag{7}$$

$$\text{where} \quad \rho = \mathrm{clip}\left(\frac{WH}{2} + \frac{16(V_T - V_L)}{V_T + V_L + 32}, 0, \frac{WH}{2}\right), \tag{8}$$

and $\mathrm{clip}(x, l, u) = \min(\max(x, l), u)$. Thus, this candidate keeps the planar philosophy, but suppresses local reference fluctuations before prediction and slightly shifts the final averaged surface according to the top-left variation imbalance.

The $\lambda = 5$ predictor instead changes how the Planar surface is formed. It first constructs a Planar-like surface using the same two components in Eqn. (2), but normalizes their sum by $W + H$ rather than using the original weights in Eqn. (1):

$$P_{\mathrm{s}}(x,y) = \frac{F_{\mathrm{h}}(x,y) + F_{\mathrm{v}}(x,y)}{W + H}. \tag{9}$$

It then computes the center-oriented references

$$\begin{aligned} C_T &= \frac{T(W/2-1,-1) + T(W/2,-1)}{2}, \\ C_L &= \frac{L(-1,H/2-1) + L(-1,H/2)}{2}, \end{aligned} \tag{10}$$

and blends the average $C = (C_T + C_L)/2$ with $P_{\mathrm{s}}(x,y)$ as

$$Q(x,y) = \frac{(32-a)P_{\mathrm{s}}(x,y) + aC}{32}, \tag{11}$$

where $a = 8$ for $WH \le 64$, $a = 4$ for $WH \le 256$, and $a = 1$ otherwise. Thus, smaller blocks receive a stronger center-reference contribution. Finally, the predictor computes the global boundary mean $\bar{R}$ and boundary range $[R_{\min}, R_{\max}]$. If $Q(x,y)$ falls inside this range, it is kept unchanged; otherwise, it is softly pulled toward $B = (C + \bar{R})/2$:

$$\hat{P}_{\lambda=5}(x,y) = \begin{cases} Q(x,y), & \text{inside}, \\ (3Q(x,y) + B)/4, & \text{otherwise}. \end{cases} \tag{12}$$

TABLE II
EVOLVED PLANAR RESULTS (ANCHOR: DEFAULT ECM-19.1).

416x240 resolution	Mode replacement BD-rate	Mode replacement ΔEncT	Mode insertion BD-rate	Mode insertion ΔEncT
BasketballDrill	−0.0020%	−1.33%	−0.0298%	+0.21%
BQMall	−0.0488%	−0.52%	−0.0005%	−3.26%
PartyScene	−0.0128%	+2.35%	−0.0184%	+2.48%
RaceHorsesC	−0.0119%	+1.87%	−0.0162%	+3.45%
BasketballPass	−0.0141%	+1.55%	−0.0595%	+3.90%
BQSquare	−0.0724%	−1.83%	−0.0163%	+0.03%
BlowingBubbles	−0.0439%	−1.51%	−0.0401%	+0.43%
RaceHorses	−0.0111%	+2.60%	−0.0723%	+0.33%
All Average	**−0.0271%**	**+0.42%**	**−0.0316%**	**+0.90%**

B. ECM Mode Replacement and Insertion

We then extend the experiment to ECM-19.1 [11], the state-of-the-art reference platform with more competitive coding tools. Based on the two directional Planar modes [12], we study two integration strategies: directly replacing the existing directional predictors and inserting a new Planar-like predictor as an additional mode. The replacement experiment follows the VVenC setting. For the insertion experiment, the generated mode is placed after the original Planar mode and before the two directional Planar modes, with its own entropy context. In the rough RDO stage, the directional-Planar candidate list is expanded to include the generated mode. For transform selection, the generated mode is treated as the original Planar.

To keep the evaluation cost manageable, we restrict the experiment to eight 416×240 sequences, including four Class D sequences and four Class C sequences downsampled to the same resolution. Training uses one frame per sequence, while testing reports the performance over ten frames. All ECM experiments run 30 iterations. The YUV-PSNR ratio is 10:1:1 [16]. The testing results are shown in Table II. Under this specific setting, both integration strategies achieve consistent coding gains. Compared with the ECM anchor, directly replacing the two directional Planar predictors gives 0.0271% BD-rate reduction with 0.42% encoding-time increase, while inserting a new Planar-like mode gives 0.0316% BD-rate reduction with 0.90% encoding-time increase. Importantly, considering that the reported BD-rate of the directional Planar for Class D is around -0.05% [12], these improvements suggest that LLMs within a generation-and-evaluation loop can find meaningful Planar-family variants in a more saturated ECM setting. Since the experiment only covers eight low-resolution sequences, we treat these results as empirical observations rather than a concrete validation.

Surprisingly, the evolved replacement predictor performs operations similar to those of the VVenC ($\lambda = 2$): the two directional predictors still keep the one-direction interpolation structure, but each branch applies filtering to its one-sided boundary before prediction. The smoothed boundary is then conditionally blended with the original boundary sample according to the local gradient: relatively flat regions use more of the three-tap $[1, 2, 1]$ filtered reference, whereas sharp local transitions are mostly preserved. Thus, the replacement mode mainly modifies the stability of the boundary signal. The inserted mode is technically different. Although the implementation is still compact, its behavior is intricate with more conditioning and clipping operations. Following the generated design idea: *"it blends global and smooth gradients using the cosine similarity confidence when sign consistency fails, instead of selecting the one with smaller magnitude"*. As a result, this makes the added mode a more aggressive gradient-conditioned Planar-like surface, in contrast to the boundary-filtering refinement found in the replacement case.

IV. CONCLUSION AND DISCUSSION

In this study, we present preliminary experiments on whether LLMs are capable of designing coding tools. The observation is encouraging: using a generation-and-evaluation loop and straightforward prompt constraints that keep the predictor lightweight and codec-compatible, LLMs can discover meaningful and competitive Planar predictors under nontrivial coding settings. Specifically, for the VVenC *faster* preset, we demonstrate that novel Planar predictors with distinct mechanisms can be obtained by controlling the selection of the top-16 candidate pool with λ. Notably, despite being evolved on natural-content sequences, the predictors still generalize reasonably well to screen content (Class F), suggesting the benefit from a diverse training set. Meanwhile, even on the highly saturated ECM platform, further bitrate savings remain achievable through both mode replacement and mode insertion under a restricted low-resolution setting.

However, the current workflow still faces a clear obstacle: evaluation is computationally too expensive. Finding an effective predictor depends heavily on exploring a large function space. For example, in our ECM experiment, nearly 2000 new candidates are evaluated before a useful predictor is observed. With parallel evaluation on 256 logic cores (AMD EPYC 9754 processors), one iteration still takes about one hour. More importantly, our results show that a predictor found at low resolution does not necessarily transfer well to higher resolutions. Similarly, a predictor effective under the simpler VVenC *faster* preset may fail under more saturated tool configurations such as *slower*. Therefore, a low-complexity in-domain exploration strategy is critical. For prediction tools, existing expert workflows often use residual signals to pre-select candidates before full encoding, and similar procedures may also benefit the LLM workflow. There are also several directions for improvement. First, the feedback context to the LLM can be richer. The current loop mainly returns the code, design idea, and sequence-level BD-rate of historical candidates, while richer information such as residual maps, content features, mode competition statistics, and failure cases could make the LLM more comprehensively informed. Second, coding tool design inherently couples tool logic, syntax, and RDO algorithms. An effective joint optimization workflow would further extend the scope of LLM-based coding tool design. Looking forward, if such a workflow becomes substantially more efficient and reliable, it may provide a new source of candidate tools for future standardization exploration.

REFERENCES

[1] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.

[2] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, "Evolution of Heuristics: Towards efficient automatic algorithm design using large language model," *arXiv preprint arXiv:2401.02051*, 2024.

[3] A. Novikov, N. Vu, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," *arXiv preprint arXiv:2506.13131*, 2025.

[4] L. He, Y. Zhang, R. Lu, M. Wang, and S. Wang, "LLM-driven heuristic frame-level quantization parameter adaptation for VVenC," *arXiv preprint arXiv:2606.20847*, 2026.

[5] G. Bjontegaard, "Calculation of average PSNR differences between RD-curves," *ITU SG16 Doc. VCEG-M33*, 2001.

[6] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Transactions on circuits and systems for video technology*, vol. 13, no. 7, pp. 560–576, 2003.

[7] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm, "Overview of the Versatile Video Coding (VVC) standard and its applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 10, pp. 3736–3764, 2021.

[8] Y. Ye, M. Karczewicz, M.-L. Champel, P. Onno, L. Zhang, X. Wang, D. Wang, Z. Lyu, Y. Huo, A. Luthra, E. François, H.-B. Teo, Y. Kidani, and S.-C. Lim, "Proposed timeline and requirements for the next generation video coding standard," ITU-T/ISO/IEC Joint Video Experts Team (JVET), Sapporo, JP, Tech. Rep. JVET-AI0247/m68403, July 2024.

[9] J. Lainema and K. Ugur, "Intra picture coding with planar representations," in *28th Picture Coding Symposium*. IEEE, 2010, pp. 198–201.

[10] A. Wieckowski, J. Brandenburg, T. Hinz, C. Bartnik, V. George, G. Hege, C. Helmrich, A. Henkel, C. Lehmann, C. Stoffers, I. Zupancic, B. Bross, and D. Marpe, "VVenC: An open and optimized VVC encoder implementation," in *Proc. IEEE International Conference on Multimedia Expo Workshops (ICMEW)*, 2021, pp. 1–2.

[11] M. Coban, R.-L. Liao, K. Naser, J. Ström, and L. Zhang, "Algorithm description of Enhanced Compression Model 19 (ECM 19)," Joint Video Experts Team (JVET), Geneva, CH, Tech. Rep. JVET-AN2025-v1, October 2025.

[12] K. Kim, D. Kim, J.-H. Son, and J. S. Kwak, "EE2-1.2: Improvements on Planar horizontal and Planar vertical mode," Joint Video Experts Team (JVET), Teleconference, Tech. Rep. JVET-AC0105-v2, January 2023.

[13] X. Wang, K. Choi, S. Ma, J. Chen, and M. Karczewicz, "CE3-related: A unified MPM list for intra mode coding," Joint Video Experts Team (JVET), Tech. Rep. JVET-N0185, 2019.

[14] Z. Xin, L. Xiang, and L. Shan, "CE6-related: Modified encoder decision for transform skip," Joint Video Exploration Team, Geneva, Tech. Rep., 19–27 March 2019, jVET-N0363.

[15] DeepSeek-AI, "DeepSeek-V4: Towards highly efficient million-token context intelligence," https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf, 2026.

[16] P. Onno, G. Laroche, B. Galmiche, L. Xu, C. Liao, Y. Lu, and F. Liang, "EE2-1.3: Clipping operation refinements for the CCCM modes," Joint Video Experts Team (JVET), Santa Eulària, ES, Tech. Rep. JVET-AP0168-v1, April 2026.